



BIND 9

Configuração, Administração e Segurança de Servidores
DNS no Linux

Prof. Uirá Ribeiro

Material de apoio do curso Certificação Linux.

Todos os direitos reservados.

Manual de BIND 9 — Configuração, Administração e Segurança de Servidores DNS

Adaptado e traduzido a partir do material LPIC-2 (202), Tópico 206 — Domain Name Server

Este manual reúne e reorganiza, em formato de livro de referência, o conteúdo dos objetivos 206.1 (Configuração básica de um servidor DNS), 206.2 (Criação e manutenção de zonas DNS) e 206.3 (Proteção de um servidor DNS) da certificação LPIC-2. O texto original, em inglês, foi traduzido, adaptado e reordenado tematicamente — partindo dos fundamentos do DNS, passando pela configuração do BIND 9 e pela criação de zonas, até chegar aos mecanismos de segurança TSIG, DNSSEC e DANE — com um cheat-sheet de comandos e um conjunto de exercícios comentados consolidados ao final.

Sumário

1. Introdução ao Domain Name System	3
2. BIND 9 e o arquivo named.conf.....	5
3. Zonas primárias, secundárias e reversas.....	8
4. Arquivos de zona e registros de recursos.....	10
5. Validação e administração do BIND.....	15
6. Consultas DNS com dig e host.....	17
7. Adicionando hosts e atualizações dinâmicas.....	19
8. Otimização de zonas e delegação.....	21
9. Servidores DNS alternativos.....	22
10. Segurança do BIND 9.....	23
11. TSIG — Transaction Signatures	26
12. DNSSEC — Fundamentos.....	28
13. Habilitando DNSSEC com KASP	30
14. DANE — Autenticação de certificados via DNS.....	33
15. Cheat-sheet de comandos.....	35
16. Exercícios comentados.....	37
17. Resumo geral.....	45

1. Introdução ao Domain Name System

1.1 — O que é o DNS

O Domain Name System (DNS) é a agenda telefônica da Internet: um sistema distribuído, hierárquico e tolerante a falhas responsável por traduzir nomes de domínio legíveis por humanos (como `example.com`) em endereços IP (como `192.0.2.1`), necessários para que os hosts se comuniquem em uma rede TCP/IP. Sem o DNS, seria preciso memorizar endereços numéricos para acessar qualquer serviço na rede.

Toda a infraestrutura de DNS é organizada em uma **estrutura de árvore hierárquica**, conhecida como o espaço de nomes de domínio (domain namespace). Essa organização é o que permite que a administração do espaço de nomes seja delegada e distribuída entre milhares de organizações independentes, em vez de centralizada em um único ponto.

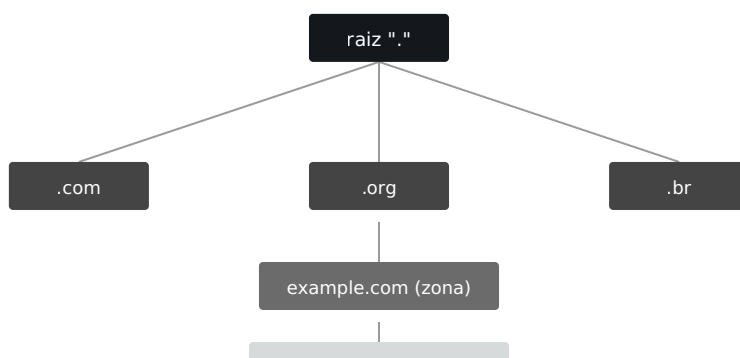


Figura 1.1 — Hierarquia do DNS: raiz, TLDs, domínio de segundo nível e subdomínios.

1.2 — Raiz, TLDs e delegação

No topo da hierarquia estão os **servidores raiz** (root servers), que são a fonte autoritativa para a zona raiz ("."). Eles não respondem diretamente por nomes como `lpi.org`, mas desempenham um papel crítico ao direcionar as consultas DNS para os servidores de top-level domain (TLD) apropriados — como `.com`, `.net` ou `.org`.

Existem atualmente 13 servidores raiz nomeados, identificados de A a M, operados por organizações independentes como ICANN, Verisign e NASA. Embora existam apenas 13 nomes lógicos, cada servidor raiz é, na prática, um sistema distribuído globalmente composto por centenas de instâncias físicas que utilizam **roteamento anycast**. Isso significa que uma consulta DNS para o F-root, por exemplo, é direcionada para a instância mais próxima, o que melhora a redundância, a velocidade e a resiliência do sistema.

Abaixo da raiz estão os top-level domains (TLDs), como `.com`, `.org` e os códigos de país, como `.br`, `.it` ou `.jp`. Abaixo destes estão os domínios de segundo nível (por

exemplo, `lpi.org`, `redhat.com`) e, potencialmente, subdomínios (como `www.example.com` ou `learning.example.com`). Essa estrutura permite delegação e administração distribuída, possibilitando que cada organização gerencie sua própria zona de forma independente.

1.3 — Zonas, servidores primários e secundários

Cada domínio ou subdomínio é gerenciado por meio de uma **zona**, que é a porção do espaço de nomes de domínio sobre a qual um determinado servidor de nomes DNS tem autoridade. Esse servidor é chamado de servidor de nomes autoritativo para aquela zona, e é ele quem fornece respostas definitivas para consultas sobre os domínios sob sua autoridade.

Quando esse servidor mantém a cópia original e gravável dos dados da zona, ele é conhecido como **servidor primário** (master), e a zona correspondente é chamada de zona primária. A zona primária contém a cópia mestre de todos os registros de recursos do domínio, normalmente armazenada em um arquivo de texto simples conhecido como arquivo de zona (zone file). Alterações nos registros DNS devem ser feitas na zona primária, e a partir dali são propagadas para os servidores secundários por meio de transferências de zona.

Um **servidor secundário** (slave) é uma cópia somente leitura do servidor primário autoritativo. Ele utiliza transferências de zona (AXFR ou IXFR) para sincronizar os dados DNS e atua como uma solução de failover ou balanceamento de carga: se o servidor primário ficar indisponível, os clientes ainda conseguem resolver nomes de domínio usando o servidor secundário.

A zona armazenada em um servidor secundário é chamada de zona secundária. Diferente da zona primária, que é mantida manualmente, a zona secundária é atualizada automaticamente por replicação a partir do servidor primário. Essa replicação garante a consistência da infraestrutura de DNS ao mesmo tempo em que distribui a carga e aumenta a redundância.

1.4 — Resolução recursiva e forwarders

Um **servidor DNS recursivo** — também chamado de resolver — atua em nome do cliente para resolver um nome de domínio do início ao fim. Quando um cliente emite uma consulta (por exemplo, `www.lpi.org`), o servidor recursivo verifica seu cache; se o registro não for encontrado, ele contata um servidor raiz, depois o servidor TLD apropriado e, por fim, o servidor autoritativo do domínio. O resultado final é devolvido ao cliente e armazenado em cache para uso futuro. Servidores recursivos costumam ser operados por provedores de acesso (ISPs) ou por redes corporativas internas.

Um **forwarder DNS** é um servidor configurado para enviar todas ou algumas consultas DNS a outro servidor DNS, em vez de resolvê-las por conta própria. Isso é especialmente útil em ambientes corporativos, onde servidores DNS internos podem

encaminhar consultas externas para um resolvidor central, como um fornecido pelo ISP ou por um serviço de DNS público. Os forwarders reduzem o tráfego externo, centralizam a aplicação de políticas e melhoram a eficiência do cache.

1.5 — Zonas reversas

Além de resolver nomes de domínio em endereços IP (lookup direto), o DNS também suporta o processo inverso: **resolver endereços IP em nomes de domínio**. Essa funcionalidade é essencial para diversos serviços de rede e mecanismos de segurança, sendo implementada por meio de um tipo especial de zona conhecida como zona reversa.

Uma zona reversa mapeia endereços IP para nomes de domínio usando registros PTR (Pointer). Em vez da notação de domínio tradicional, as zonas reversas são organizadas sob os domínios especiais `in-addr.arpa`, para IPv4, e `ip6.arpa`, para IPv6. Por exemplo, o endereço IP `192.0.2.25` seria representado em uma zona reversa como `25.2.0.192.in-addr.arpa`, e um registro PTR mapearia esse nome para um hostname como `host.example.com`.

Zonas reversas são amplamente usadas na validação de e-mail (por exemplo, em regras de SPF e filtros anti-spam), em troubleshooting de rede e em auditorias de segurança. Elas são configuradas de forma semelhante às zonas diretas, mas com a direção do mapeamento e a estrutura dos nomes de domínio invertidas.

2. BIND 9 e o arquivo `named.conf`

2.1 — O que é o BIND

O **BIND** (Berkeley Internet Name Domain) é o software de servidor DNS mais utilizado em sistemas Unix-like. A versão 9.x é o padrão atual e oferece uma implementação flexível, robusta e segura dos serviços de DNS, suportando recursos como DNSSEC, TSIG, views e ACLs.

Desenvolvido e mantido pelo Internet Systems Consortium (ISC), é uma solução de DNS completa e altamente configurável, suportando tanto o papel de servidor autoritativo quanto o de servidor recursivo. O BIND 9 é conhecido por sua configuração modular, controles de acesso poderosos e recursos avançados de DNS, capazes de atender tanto pequenos ambientes quanto redes corporativas de larga escala.

2.2 — Estrutura do arquivo `named.conf`

O arquivo de configuração central do BIND é o `/etc/named.conf` (ou, em sistemas baseados em Debian, geralmente `/etc/bind/named.conf`). Esse arquivo define configurações globais, políticas de segurança e referências a outros arquivos de configuração, como definições de zona ou opções de logging.

O arquivo é estruturado em **statements** delimitados por chaves ({ statement; } ;) — cada statement dentro do bloco termina com ponto e vírgula (;), e também é colocado um ponto e vírgula logo após a chave de fechamento de cada bloco. Essa estrutura de chaveamento aninhado, com pontos e vírgulas em cada nível, aparece com frequência nas provas de certificação e é a base para ler e escrever qualquer configuração do BIND. Os principais blocos são:

- O bloco `options` : define o comportamento global do servidor.
- O bloco `zone` : declara zonas DNS específicas que o servidor irá gerenciar.
- O statement `include` : permite modularizar a configuração em múltiplos arquivos.
- O bloco `logging` : define como e onde o BIND registra suas mensagens operacionais.

2.3 — O bloco options

O statement `options` no arquivo `named.conf` é usado para definir o comportamento global do servidor. Ele especifica parâmetros como o diretório onde os arquivos de zona são armazenados, se o servidor permite consultas recursivas, quais clientes têm permissão para enviar consultas, além de diversas configurações de desempenho e segurança. Essa seção se aplica ao servidor como um todo e forma o comportamento fundamental do daemon `named`.

2.4 — O bloco zone e ACLs

O statement `zone` é responsável por declarar as zonas DNS específicas que o servidor irá gerenciar. Ele indica se o servidor atua como primário (master) ou secundário (slave) para cada zona, e associa cada zona a um arquivo de zona correspondente, que contém os registros DNS. Para zonas diretas, esse arquivo mapeia nomes de domínio para endereços IP; para zonas reversas, ele lida com o mapeamento inverso, de endereços IP para nomes de domínio.

Uma ACL (Access Control List) é uma lista nomeada de endereços ou redes que pode ser referenciada em diretivas como `allow-query`, `allow-transfer` ou `allow-recursion`, permitindo aplicar a mesma política de acesso de forma consistente em vários pontos da configuração.

2.5 — Modularização com include e logging

Para melhorar a clareza e a facilidade de manutenção, o statement `include` é usado para modularizar os arquivos de configuração. Em vez de manter um único arquivo de configuração extenso, os administradores podem colocar listas de controle de acesso (ACLs), chaves DNSSEC ou definições de views em arquivos separados e referenciá-los por meio de statements `include`. Essa abordagem simplifica a manutenção e permite uma estrutura de configuração mais organizada.

O statement `logging` define como e onde o BIND registra suas mensagens operacionais. Os administradores podem criar múltiplos channels de log (como `file` ou `syslog`) e associá-los a categorias específicas, como queries, security ou update. Esse sistema de logging granular facilita o monitoramento do comportamento do DNS, a detecção de anomalias e o troubleshooting em ambientes de produção.

Exemplo de `named.conf` de um servidor DNS autoritativo:

```
options {
    listen-on port 53 { any; };
    listen-on-v6 port 53 { any; };
    directory "/var/named";
    recursion no;
    allow-query { any; };
    allow-transfer { trusted-slaves; };
};

acl "trusted-slaves" {
    192.0.2.10;
    192.0.2.11;
};

zone "." IN {
    type hint;
    file "named.ca";
};

zone "example.com" IN {
    type master;
    file "example.com.zone";
    allow-transfer { trusted-slaves; };
};

zone "2.0.192.in-addr.arpa" IN {
    type master;
    file "example.com.rev";
};
```

Nesse bloco `options`, `listen-on port 53 { any; };` faz o servidor escutar em todas as interfaces IPv4, e `listen-on-v6 port 53 { any; };` faz o mesmo para todas as interfaces IPv6, garantindo que o servidor seja acessível a partir de qualquer interface de rede configurada no host.

A diretiva `directory "/var/named";` define o diretório de trabalho onde o BIND procura os arquivos de zona e armazena dados de runtime. A opção `recursion no;` desabilita consultas recursivas, tornando o servidor apenas autoritativo (authoritative-only) — apropriado para servidores DNS públicos que não devem resolver domínios externos em nome de clientes.

A diretiva `allow-query { any; };` permite que qualquer cliente realize consultas DNS, o que é típico em servidores DNS públicos; para um DNS apenas interno, isso deveria ser restrito a redes confiáveis. Uma ACL chamada `"trusted-slaves"` é definida para

listar os endereços IP dos servidores secundários autorizados a solicitar transferências de zona, e essa ACL é referenciada na diretiva `allow-transfer` dentro de cada zona, oferecendo replicação controlada dos dados de zona.

Cada bloco `zone` declara uma zona DNS que o servidor gerencia como master. A zona direta `example.com` e a zona reversa `2.0.192.in-addr.arpa` estão associadas aos seus respectivos arquivos de zona e protegidas restringindo as transferências apenas aos servidores secundários confiáveis definidos anteriormente.

3. Zonas primárias, secundárias e reversas

3.1 — Declarando uma zona primária (master)

Em uma instalação típica do BIND 9.x, os arquivos de zona são armazenados em um diretório de trabalho padrão, definido pela diretiva `directory` dentro do bloco `options`. Na maioria dos sistemas baseados em Red Hat, o local padrão é `/var/named`, enquanto em sistemas baseados em Debian é comumente `/etc/bind`.

Os arquivos de zona precisam ser acessíveis pelo daemon `named`, que geralmente é executado com um usuário restrito (por exemplo, `named` ou `bind`, conforme a distribuição) e pode estar confinado por ambientes chroot ou por políticas de AppArmor/SELinux. Normalmente, os arquivos de zona pertencem a esse usuário e grupo restritos, com permissões de leitura/gravação limitadas.

Uma declaração de zona primária em `named.conf` tem a seguinte forma:

```
zone "example.com" IN {
    type master;
    file "example.com.zone";
};
```

Nesse exemplo, o BIND procurará pelo arquivo `example.com.zone` dentro do caminho `directory` padrão (por exemplo, `/var/named/example.com.zone`). Se os arquivos de zona estiverem armazenados fora desse caminho padrão, seus caminhos absolutos completos devem ser fornecidos, e as permissões de leitura apropriadas devem ser concedidas.

3.2 — Zonas secundárias e transferência de zona

Uma zona secundária é uma réplica somente leitura de uma zona primária, obtida por meio de transferências de zona. Ela melhora a redundância e o balanceamento de carga. Veja como uma zona secundária é declarada em `named.conf`:

```
zone "example.com" IN {  
    type slave;  
    file "slaves/example.com.zone";  
    masters { 192.0.2.1; };  
};
```

Nessa configuração, o servidor contatará o primário em `192.0.2.1` para solicitar a cópia mais recente da zona `example.com`. O arquivo será armazenado localmente em `slaves/example.com.zone`. A diretiva `file` indica o caminho (relativo ao `directory` definido em `options`); esse arquivo é gerenciado automaticamente pelo BIND e não deve ser editado manualmente.

O BIND suporta dois métodos principais para transferir dados de zona DNS entre servidores: a Asynchronous Full Transfer (**AXFR**) é o método tradicional, que transfere o arquivo de zona inteiro do primário para o secundário — simples e confiável, mas ineficiente para zonas grandes, já que cada alteração dispara uma transferência completa. A Incremental Zone Transfer (**IXFR**) é um método mais eficiente, que transfere apenas as mudanças feitas desde a última atualização. Ambos os tipos de transferência usam **TCP**.

3.3 — allow-transfer e segurança de transferência

A transferência de zona deve ser explicitamente permitida usando a diretiva `allow-transfer`, que deve ser definida dentro da configuração da zona primária no servidor DNS autoritativo primário. Esse statement controla quais servidores remotos têm permissão para realizar transferências de zona daquela zona específica; é uma medida de segurança crítica, já que transferências irrestritas poderiam expor todos os registros DNS a entidades não autorizadas, tornando o servidor vulnerável a coleta de dados (data harvesting) e ataques de reconhecimento.

Um exemplo típico usando um endereço IP específico:

```
zone "example.com" IN {  
    type master;  
    file "example.com.zone";  
    allow-transfer { 192.0.2.10; };  
};
```

Nesse exemplo, apenas o servidor em `192.0.2.10` está autorizado a solicitar transferências de zona para `example.com`.

3.4 — Zonas reversas em `named.conf`

Uma zona reversa mapeia endereços IP de volta a nomes de domínio usando registros `PTR` — o oposto do lookup direto mais comum, que resolve nomes de domínio em endereços IP. Esse processo é conhecido como reverse DNS lookup, e é

essencial em áreas como troubleshooting de rede, logging, verificação de e-mail e auditoria de segurança.

Para a rede IPv4 `192.0.2.0/24`, a zona reversa seria definida da seguinte forma:

```
zone "2.0.192.in-addr.arpa" IN {  
    type master;  
    file "192.0.2.rev";  
};
```

4. Arquivos de zona e registros de recursos

4.1 — Estrutura de um arquivo de zona e o registro SOA

Arquivos de zona são arquivos de texto simples que definem os registros de recursos DNS para um domínio específico (zona direta) ou bloco de IPs (zona reversa). Um arquivo de zona geralmente começa com uma diretiva `$TTL` (Time To Live), seguida por um registro `SOA` (Start of Authority).

O **TTL** define por quanto tempo um registro DNS pode ficar em cache por resolvers e clientes antes de precisar ser atualizado junto ao servidor autoritativo, sendo especificado em segundos. O cifrão (`$`) em `$TTL` é uma diretiva usada em arquivos de zona para definir comportamento global, e não um registro de recurso individual. É comum que um arquivo de zona comece dessa forma apenas por conveniência — para evitar repetir um TTL em cada linha — mas isso não é obrigatório, já que cada registro pode ter seu próprio TTL.

Exemplo de um arquivo de zona direta completo para `example.com`:

```
$TTL 86400
@   IN  SOA ns1.example.com. admin.example.com. (
        2025080201 ; Serial
        3600       ; Refresh
        1800       ; Retry
        1209600    ; Expire
        86400 )    ; Minimum TTL

        IN  NS   ns1.example.com.
        IN  NS   ns2.example.com.

ns1  IN  A       192.0.2.1
ns2  IN  A       192.0.2.2

www   IN  A       192.0.2.10
www6  IN  AAAA    2001:db8::10
ftp   IN  CNAME   www.example.com.
mail  IN  A       192.0.2.20

@     IN  MX  10   mail.example.com.
@     IN  TXT   "v=spf1 ip4:192.0.2.20 include:_spf.example.com ~all"
```

Cada registro DNS em um arquivo de zona costuma seguir um formato estruturado, composto por cinco campos:

```
[nome] [TTL] [classe] [tipo] [dado]
```

O primeiro campo, `nome`, especifica o nome de domínio ao qual o registro se aplica. Se o `nome` for omitido, ele herda o valor do registro anterior. Alternativamente, o símbolo `@` pode ser usado para representar a origem da zona, tipicamente o domínio declarado na definição da zona.

O campo `TTL` é opcional; quando não é explicitamente informado, aplica-se o valor padrão definido pela diretiva `$TTL` no topo do arquivo de zona. O campo `classe` indica a família de protocolo à qual o registro pertence — na prática, quase sempre `IN`, que significa Internet, a classe mais usada em DNS moderno.

O campo `tipo` define o tipo de registro DNS que está sendo criado — exemplos incluem `A`, `AAAA`, `MX`, `NS`, `TXT`, `CNAME` e `PTR`. O campo `dado` contém o conteúdo real ou o destino do registro; dependendo do tipo, pode ser um endereço IP (para `A` ou `AAAA`), um hostname (para `CNAME` ou `MX`) ou uma string de texto (para registros `TXT`).

```
www 3600 IN A 192.0.2.10
```

Nesse registro, `www` é o nome, `3600` é o TTL em segundos, `IN` é a classe, `A` é o tipo de registro e `192.0.2.10` é o dado (endereço IPv4).

O registro **SOA** marca o início de um arquivo de zona DNS e contém informações administrativas e de temporização para a zona. Ele identifica o servidor de nomes autoritativo primário, a parte responsável (geralmente o e-mail do administrador) e diversos parâmetros que controlam as transferências de zona e o comportamento de cache:

```
@ IN SOA ns1.example.com. admin.example.com. (
    2025080201 ; serial
    3600      ; refresh
    1800      ; retry
    1209600   ; expire
    86400     ; minimum
```

- **ns1.example.com.** : o principal servidor de nomes autoritativo da zona.
- **admin.example.com.** : o e-mail do administrador da zona, com o **@** substituído por um ponto (equivale a **admin@example.com**).
- **serial** : um número de série incrementado a cada atualização da zona.
- **refresh** : com que frequência os servidores secundários verificam se há atualizações (em segundos).
- **retry** : com que frequência tentar novamente caso o refresh falhe.
- **expire** : quando a zona deixa de ser autoritativa caso o refresh falhe repetidamente.
- **minimum** : o TTL padrão para respostas negativas.

Comentários em arquivos de zona começam com ponto e vírgula (**;**) e se estendem até o final da linha; são ignorados pelo servidor DNS e costumam ser usados para anotar registros e esclarecer seções do arquivo.

O **número de série** funciona como um indicador de versão da zona. Sempre que o arquivo de zona é modificado — por exemplo, ao adicionar, remover ou alterar um registro — o número de série precisa ser incrementado. O motivo é simples: os servidores secundários (slave) usam o número de série para determinar se a zona mudou. Deixar de atualizar o serial faz com que os secundários não busquem os registros atualizados, levando a inconsistências e dados DNS desatualizados em toda a infraestrutura.

Uma convenção comum é usar o formato **YYYYMMDDnn** (ano, mês, dia, revisão), como em **2025080201** (2 de agosto de 2025, primeira edição do dia).

4.2 — NS, A e AAAA

O registro **NS** (Name Server) identifica os servidores DNS autoritativos de uma zona — servidores responsáveis por responder consultas sobre o domínio. Toda zona deve ter ao menos um registro **NS**, embora seja fortemente recomendado definir dois ou mais para redundância. Cada registro **NS** especifica o nome de domínio de um

servidor de nomes, que precisa resolver para um endereço IP por meio de um registro `A` ou `AAAA` em algum lugar da zona ou de uma zona pai.

```
@    IN  NS  ns1.example.com.
@    IN  NS  ns2.example.com.
```

Nesse exemplo, o símbolo `@` se refere à origem da zona atual (por exemplo, `example.com.`), e os dois servidores listados (`ns1.example.com.` e `ns2.example.com.`) são declarados como autoritativos para o domínio.

O registro `A` (Address) mapeia um nome de domínio para um endereço IPv4 em notação decimal com pontos:

```
www   IN  A  192.0.2.10
mail  IN  A  192.0.2.20
```

Se o símbolo `@` for usado como nome, ele se aplica à própria origem do domínio (por exemplo, `example.com.`). Múltiplos registros `A` também podem ser usados para o mesmo nome, permitindo balanceamento de carga entre diferentes endereços IP.

O registro `AAAA` (Quad-A) mapeia um nome de domínio para um endereço IPv6, em notação hexadecimal com dois-pontos:

```
www   IN  AAAA  2001:db8::10
mail  IN  AAAA  2001:db8::20
```

4.3 — MX, CNAME e TXT

O registro `MX` (Mail Exchange) define os servidores de e-mail responsáveis por receber mensagens em nome de um domínio. Cada registro `MX` inclui um valor de prioridade (também chamado de preferência) e um nome de domínio totalmente qualificado (FQDN) do servidor de e-mail. Valores de prioridade menores indicam maior preferência — o servidor de e-mail com o menor número será tentado primeiro; se ele estiver indisponível, o próximo menor será usado, o que oferece uma capacidade básica de failover.

```
@    IN  MX  10  mail1.example.com.
@    IN  MX  20  mail2.example.com.
```

Nesse exemplo, o e-mail para `example.com` será entregue primeiro a `mail1.example.com.`; se falhar, a entrega recorrerá ao servidor `mail2.example.com.` Todo servidor de e-mail especificado em um registro `MX` também precisa ter um registro `A` ou `AAAA` correspondente para resolver seu nome em um endereço IP. Note que registros `CNAME` não devem ser usados para servidores de e-mail em entradas `MX`.

O registro `CNAME` (Canonical Name) cria um alias que mapeia um nome de domínio para outro. É usado quando se deseja que vários nomes de domínio apontem para o mesmo recurso, sem duplicar registros `A` ou `AAAA`. Quando um resolver DNS encontra um registro `CNAME`, ele substitui o nome consultado pelo nome canônico (alvo) e reinicia o processo de resolução; a resposta final é o endereço IP associado ao nome canônico.

```
ftp    IN    CNAME  www.example.com.  
blog   IN    CNAME  host.example.net.
```

Nesse exemplo, `ftp.example.com` resolve para o mesmo endereço que `www.example.com`, e `blog.example.com` é um alias para `host.example.net`.

O registro `TXT` (Text) é usado para associar texto legível por humanos, ou interpretável por máquinas, a um nome de domínio. É comumente usado para verificação de propriedade de domínio, configurações de segurança de e-mail (como SPF, DKIM e DMARC) e outros metadados personalizados exigidos por serviços externos. Um único domínio pode ter múltiplos registros `TXT`, e cada registro pode conter até 255 caracteres por string — para conteúdo mais longo, múltiplas strings entre aspas podem ser usadas na mesma linha.

```
@      IN    TXT    "v=spf1 include:_spf.example.com ~all"  
mail   IN    TXT    "google-site-verification=abc123xyz"
```

Nesse exemplo, o primeiro registro define uma regra SPF (Sender Policy Framework) para o domínio, ajudando servidores de e-mail a identificar remetentes legítimos. O segundo registro é frequentemente usado para verificar a propriedade do domínio junto a serviços como o Google.

4.4 — FQDNs e a ambiguidade de origem

Em arquivos de zona DNS, nomes de domínio que não terminam com um ponto final (`.`) são considerados relativos à origem da zona atual. Isso significa que o servidor DNS acrescenta automaticamente a origem (definida pelo símbolo `@` ou pela declaração da zona) a qualquer nome que não termine em ponto. Por exemplo, em um arquivo de zona para `example.com`, o registro `www IN CNAME host` será interpretado como `www.example.com. IN CNAME host.example.com.`

Porém, se a intenção era apontar para um domínio externo, como `host.otherdomain.com`, omitir o ponto final causaria um erro: `www IN CNAME host.otherdomain.com` se tornaria incorretamente `host.otherdomain.com.example.com`. Para evitar essa ambiguidade, use sempre um nome de domínio totalmente qualificado (FQDN) — um nome de domínio completo terminado em ponto — o que instrui o servidor DNS a tratar o nome como absoluto, sem acrescentar a origem:

```
www IN CNAME host.otherdomain.com.
```

4.5 — Zonas reversas e o registro PTR

Zonas reversas são definidas usando domínios especiais: `in-addr.arpa` para IPv4 e `ip6.arpa` para IPv6. O endereço IP é invertido e combinado com o sufixo apropriado para formar o nome da zona; por exemplo, o endereço IPv4 `192.0.2.25` pertence à zona reversa `2.0.192.in-addr.arpa`.

Exemplo de um arquivo de zona reversa com registros PTR:

```
$TTL 86400
@ IN SOA ns1.example.com. admin.example.com. (
    2025080201 ; Serial
    3600       ; Refresh
    1800       ; Retry
    1209600    ; Expire
    86400      ; Minimum TTL

    IN NS ns1.example.com.

    1 IN PTR ns1.example.com.
    2 IN PTR ns2.example.com.
    10 IN PTR www.example.com.
    25 IN PTR mail.example.com.
```

Cada registro `PTR` corresponde ao último octeto do endereço IP sendo mapeado. Por exemplo, a entrada `25 IN PTR mail.example.com.` significa que `192.0.2.25` resolverá para `mail.example.com` durante uma consulta reversa.

5. Validação e administração do BIND

5.1 — `named-checkconf`

O utilitário `named-checkconf` é uma ferramenta essencial fornecida pelo BIND para validar a sintaxe e a estrutura do arquivo de configuração `named.conf`, além de todos os arquivos que ele inclui. Ele garante que a configuração esteja livre de erros de sintaxe antes de iniciar ou recarregar o servidor DNS, ajudando os administradores a identificar erros cedo e evitar falhas em produção.

O uso mais comum é executar o comando sem argumentos, o que verifica o arquivo de configuração padrão (geralmente `/etc/named.conf` ou `/etc/bind/named.conf`):

```
$ sudo named-checkconf
```

Se a configuração for válida, o comando não retorna nenhuma saída e encerra silenciosamente. Se houver um erro, ele imprime uma mensagem com o nome do arquivo e o número da linha em que o problema ocorreu:

Exemplo de saída de erro:

```
$ sudo named-checkconf
/etc/named.conf:13: missing ';' before '}'
```

Também é possível especificar um caminho customizado para um arquivo de configuração:

```
$ sudo named-checkconf /path/to/custom-named.conf
```

A opção `-z` também verifica os arquivos de zona definidos em `named.conf`. A opção `-p` imprime a configuração resultante em forma canônica:

```
$ sudo named-checkconf -z -p
```

Esse comando é especialmente útil para validar não apenas a sintaxe, mas também se os arquivos de zona referenciados existem e são legíveis pelo usuário sob o qual o `named` é executado.

5.2 — named-checkzone

O utilitário `named-checkzone` é usado para verificar a correção de um arquivo de zona DNS antes que ele seja carregado pelo servidor BIND. Essa ferramenta verifica a sintaxe, a estrutura e a consistência dos registros dentro do arquivo, incluindo a presença de um registro `SOA` válido, a formatação correta dos registros de recursos e o uso correto dos números de série.

A sintaxe do comando é:

```
$ sudo named-checkzone <nome-da-zona> <arquivo-de-zona>
```

Exemplo:

```
$ sudo named-checkzone example.com /var/named/example.com.zone
zone example.com/IN: loaded serial 2025080201
OK
```

Se o arquivo de zona for válido, a ferramenta imprime o nome da zona, o tipo (IN para Internet), o número de série e um status `OK`. Se forem encontrados erros, eles são reportados com números de linha e descrições detalhadas.

5.3 — Administrando com rndc

O utilitário `rndc` (Remote Name Daemon Control) oferece controle administrativo via linha de comando sobre o servidor BIND em execução. Ele se comunica com o daemon `named` por meio de um canal seguro definido por uma chave de autenticação, permitindo que os administradores executem tarefas comuns — como recarregar zonas, limpar caches e verificar o status do servidor — sem reiniciar o serviço.

Usar o `rndc` evita indisponibilidade ao aplicar mudanças de configuração ou atualizações em arquivos de zona. Por padrão, o `rndc` usa a chave e as configurações definidas em `/etc/rndc.conf`, ou por meio dos statements `controls` e `key` dentro de `named.conf`. Uma configuração simples e segura pode ser gerada com:

```
$ sudo rndc-confgen -a
```

Isso cria uma chave compartilhada em `/etc/rndc.key`, referenciada tanto pelo `rndc` quanto pelo `named`. Para garantir que o `named` carregue essa chave, adicione o seguinte em `named.conf`:

```
include "/etc/rndc.key";

controls {
    inet 127.0.0.1 port 953
        allow { 127.0.0.1; } keys { "rndc-key"; };
};
```

Isso permite que o `rndc` controle o `named` pela porta TCP 953 em localhost, usando a chave gerada.

Comandos rndc comuns:

Comando	Função
<code>rndc reload</code>	Recarrega toda a configuração e todas as zonas.
<code>rndc reload <zona></code>	Recarrega uma zona específica.
<code>rndc status</code>	Exibe o status do servidor.
<code>rndc flush</code>	Limpa todos os caches.
<code>rndc stop</code>	Para o servidor de forma graciosa.

6. Consultas DNS com dig e host

Ao trabalhar com um servidor DNS BIND, duas ferramentas essenciais para testar e diagnosticar o comportamento do DNS são `dig` e `host`. Ambas são utilitários cliente poderosos que consultam servidores DNS diretamente, sendo amplamente

usadas por administradores para verificar configurações de zona, checar resolução e depurar problemas.

6.1 — dig (Domain Information Groper)

O comando `dig` é uma ferramenta de consulta DNS flexível, que fornece saída detalhada e é ideal para testar respostas DNS. Ele mostra não apenas a seção de resposta, mas também informações de autoridade, TTLs e tempos de consulta. O `dig` é amplamente usado para verificar se um servidor BIND está resolvendo domínios corretamente e servindo dados autoritativos.

Consultando um registro A:

```
$ dig @127.0.0.1 example.com A
```

Esse comando envia uma consulta DNS diretamente ao servidor BIND local (127.0.0.1), solicitando o registro A de `example.com`, que corresponde ao endereço IPv4 do domínio.

Consultando um servidor DNS específico:

```
$ dig @192.0.2.1 www.example.com +noall +answer
www.example.com.300      IN      CNAME   www.example.com-v4.edgesuite.net.
www.example.com-v4.edgesuite.net. 21600 IN CNAME a1422.dscr.akamai.net.
a1422.dscr.akamai.net.  20      IN      A        201.17.166.170
a1422.dscr.akamai.net.  20      IN      A        201.17.166.171
```

As opções `+noall +answer` alteram a saída para exibir apenas a seção de resposta, omitindo metadados adicionais para maior clareza.

Consultando os registros NS de uma zona:

```
$ dig example.com NS
```

6.2 — host

O comando `host` é uma alternativa mais simples ao `dig`, focada em retornar rapidamente respostas a consultas específicas. É ideal para consultas rápidas, quando não se precisa de todos os detalhes técnicos exibidos pelo `dig`.

Consultando um registro A:

```
$ host example.com
example.com has address 96.7.128.175
```

Consultando um servidor de nomes específico:

```
$ host www.example.com 1.1.1.1
```

7. Adicionando hosts e atualizações dinâmicas

Adicionar um novo host a uma zona DNS envolve criar registros correspondentes tanto no arquivo de zona direto quanto no reverso, para garantir uma resolução de nomes e consultas reversas consistentes. Há duas formas principais de adicionar um novo host a uma zona DNS usando o BIND: editar manualmente os arquivos de zona, ou realizar uma atualização dinâmica segura usando o utilitário `nsupdate`.

7.1 — Adição manual em zonas forward e reverse

Para adicionar um novo host manualmente, é preciso incluir um registro `A` ou `AAAA` no arquivo da zona direta:

```
ftp IN A      192.0.2.55
ftp IN AAAA   2001:db8::55
```

E adicionar o registro `PTR` correspondente no arquivo da zona reversa:

```
55 IN PTR ftp.example.com.
```

Após adicionar esses registros, é importante incrementar o número de série do SOA em ambos os arquivos de zona, e recarregar as zonas com o `rndc`:

```
$ sudo rndc reload example.com
$ sudo rndc reload 2.0.192.in-addr.arpa
```

7.2 — nsupdate e TSIG (DNS dinâmico)

O BIND também suporta atualizações dinâmicas de DNS por meio da ferramenta `nsupdate`, que permite adicionar ou remover registros sem editar diretamente os arquivos de zona. A zona precisa ser configurada com a diretiva `allow-update` usando uma chave TSIG, que autoriza de forma segura as atualizações dinâmicas — sem essa configuração, tentativas de modificar a zona com ferramentas como `nsupdate` serão rejeitadas pelo servidor DNS.

Para gerar uma chave TSIG (segredo compartilhado), use o utilitário `tsig-keygen`:

```
$ tsig-keygen mykey
key "mykey" {
    algorithm hmac-sha256;
    secret "7/5QMsuGmt999B0kvZnsY/KM50oW9mmshB91iYeyV9g=";
};
```

Em seguida, copie o bloco de chave gerado para o seu `named.conf`. Para autorizar atualizações em uma zona específica usando essa chave, adicione a diretiva `allow-update` dentro do bloco `zone` correspondente:

```
key "mykey" {
    algorithm hmac-sha256;
    secret "7/5QMsuGmt999B0kvZnsY/KM50oW9mmshB91iYeyV9g=";
};

zone "example.com" IN {
    type master;
    file "example.com.zone";
    allow-update { key "mykey"; };
};
```

Após reiniciar o `named`, é possível adicionar dinamicamente um novo host usando o utilitário `nsupdate`. Primeiro, crie um arquivo chamado `update.txt` contendo os comandos de atualização — abordagem útil para scripts ou atualizações em lote:

```
server 127.0.0.1
key hmac-sha256:mykey 7/5QMsuGmt999B0kvZnsY/KM50oW9mmshB91iYeyV9g=
update add ftp.example.com. 3600 IN A 192.0.2.55
send
```

Em seguida, execute:

```
$ nsupdate -v update.txt
```

Se a atualização for bem-sucedida, os registros são imediatamente adicionados, e o servidor cuida automaticamente do incremento do serial e da gravação da zona.

7.3 — Journal files (.jnl) e sincronização

Quando o `nsupdate` é usado para modificar registros DNS dinamicamente, o BIND não altera diretamente o arquivo de texto da zona original (por exemplo, `example.com.zone`). Em vez disso, ele gerencia a zona por meio de um arquivo journal (com extensão `.jnl`), que registra as mudanças em formato binário.

Esse arquivo journal armazena as mudanças incrementais feitas na zona, e é periodicamente mesclado ao arquivo de zona principal sob condições específicas — como quando o servidor é reiniciado, ou quando comandos como `rndc sync`, `rndc freeze` ou `rndc thaw` são emitidos. Esse mecanismo garante a consistência entre as atualizações dinâmicas e o arquivo de zona estático, mantendo a disponibilidade do serviço DNS.

8. Otimização de zonas e delegação

8.1 — masterfile-format e named-compilezone

A opção `masterfile-format` define como um arquivo de zona é armazenado e lido pelo BIND 9. Por padrão, os arquivos de zona são escritos em formato texto, legível e fácil de editar. No entanto, o BIND também suporta um formato raw (binário), mais eficiente para ser interpretado pelo servidor DNS — particularmente útil para zonas grandes ou ambientes de alto desempenho.

O utilitário `named-compilezone` pode gerar uma versão compilada de um arquivo de zona. Esse formato binário melhora o desempenho ao reduzir significativamente o tempo de parsing durante a inicialização do BIND, especialmente para zonas grandes. Para converter uma zona como `example.com` para usar formato binário, use o seguinte comando:

```
$ sudo named-compilezone -f text -F raw -o /var/named/db.example.com.nzf
example.com /var/named/example.com.zone
```

Onde: `-f text` especifica o formato de entrada (padrão); `-F raw` define o formato de saída como binário (raw); `-o` designa o arquivo compilado de saída; e os dois últimos argumentos são o nome do domínio e o caminho para o arquivo de zona original.

Em seguida, atualize a declaração da zona em `named.conf` para usar o formato binário:

```
zone "example.com" IN {
    type master;
    file "/var/named/db.example.com.nzf";
    masterfile-format raw;
};
```

Recarregue o BIND para aplicar a configuração de zona atualizada:

```
$ sudo rndc reload
```

8.2 — Delegando zonas a outro servidor

Delegar uma zona DNS significa atribuir autoridade sobre um subdomínio a um servidor DNS diferente. Isso permite que a infraestrutura de DNS escale e distribua a responsabilidade administrativa — a delegação é comumente usada em grandes organizações, ISPs e ambientes multi-tenant. A delegação de zona permite distribuir a administração de um subdomínio (zona filha) para outro servidor DNS, enquanto a zona pai permanece autoritativa apenas para seus próprios registros.

Para isso, adicione registros NS na zona pai apontando para os servidores DNS autoritativos do subdomínio delegado — essa etapa estabelece a delegação, informando aos resolvers quais servidores são responsáveis pelo subdomínio. Por exemplo, para delegar a zona `sub.example.com` aos servidores de nomes `ns1.sub.example.com` e `ns2.sub.example.com`, adicione os seguintes registros à zona `example.com`:

```
sub.example.com. IN NS ns1.sub.example.com.
sub.example.com. IN NS ns2.sub.example.com.
```

Certifique-se de que `ns1.sub.example.com` e `ns2.sub.example.com` tenham registros A ou AAAA correspondentes na zona pai, ou sejam resolvíveis por DNS externo, para que os resolvers consigam alcançá-los:

```
ns1.sub.example.com. IN A 192.0.2.53
ns2.sub.example.com. IN A 192.0.2.54
```

No servidor DNS responsável por `sub.example.com`, crie um arquivo de zona separado e configure-o em `named.conf`:

```
zone "sub.example.com" IN {
    type master;
    file "/var/named/db.sub.example.com";
};
```

Esse arquivo de zona deve conter os registros SOA, NS e demais registros de recursos do subdomínio. Isso possibilita uma administração de DNS hierárquica e distribuída, essencial para escalabilidade e delegação de autoridade.

9. Servidores DNS alternativos

Embora o BIND seja o servidor DNS mais usado e rico em recursos em sistemas Unix-like, soluções alternativas de DNS, como dnsmasq e PowerDNS, oferecem vantagens distintas para casos de uso específicos.

9.1 — dnsmasq

O dnsmasq é um forwarder DNS leve, com servidor DHCP embutido, projetado para simplicidade e baixo consumo de recursos. É ideal para redes pequenas, sistemas embarcados, ambientes virtuais e containers. O dnsmasq consegue resolver hostnames locais, encaminhar consultas a servidores DNS upstream (como os de um ISP ou resolvers públicos) e armazenar respostas em cache para acesso mais rápido. Sua configuração é mínima, tipicamente usando um único arquivo `/etc/dnsmasq.conf` ou opções de linha de comando.

O `dnsmasq` é particularmente adequado para redes domésticas e de pequenos escritórios (SOHO), devido à sua natureza leve e facilidade de configuração. Ele fornece serviços de DNS e DHCP em um único pacote, o que o torna ideal para ambientes que exigem configuração mínima e baixo consumo de recursos. Além disso, o `dnsmasq` se integra perfeitamente com o DHCP, permitindo a resolução DNS automática de hostnames atribuídos dinamicamente, sem a necessidade de gerenciamento manual de arquivos de zona.

Também é comumente usado em ambientes containerizados, como Docker ou LXC, onde a implantação rápida e os namespaces de rede isolados se beneficiam de um forwarder DNS leve. Como um proxy DNS local com cache, o `dnsmasq` melhora a velocidade de resolução DNS e reduz o tráfego upstream.

9.2 — PowerDNS

O PowerDNS é um pacote de servidor DNS poderoso e modular, que suporta os modos de operação autoritativo e recursivo. É conhecido por seu alto desempenho, por seus backends de banco de dados (como MySQL, PostgreSQL e SQLite3) e por suas APIs para atualizações dinâmicas e integração com infraestrutura moderna. O PowerDNS é comumente usado em ISPs, provedores de hospedagem e ambientes corporativos onde escalabilidade e automação são essenciais.

O PowerDNS é especialmente útil em cenários em que escalabilidade, automação e integração com sistemas externos são prioridades. Um caso de uso comum é como servidor DNS autoritativo para ISPs, onde alto volume de consultas e desempenho são críticos. Sua capacidade de conectar-se a backends SQL, como MySQL ou PostgreSQL, permite o gerenciamento dinâmico de zonas diretamente a partir de um banco de dados, simplificando atualizações e provisionamento em ambientes de grande porte.

Outro recurso poderoso do PowerDNS é o suporte a APIs DNS, que possibilita integração perfeita com ferramentas de orquestração e painéis de controle — tornando-o uma escolha preferida para provedores de nuvem e plataformas de hospedagem que exigem gerenciamento automatizado de DNS. Além disso, o componente PowerDNS Recursor pode ser implantado como um resolutor recursivo escalável, oferecendo recursos avançados como limitação de taxa de consultas e Response Policy Zones (RPZ) para filtragem e reforço de segurança baseados em DNS.

10. Segurança do BIND 9

Proteger um servidor BIND 9 começa por estruturar corretamente seus arquivos de configuração e limitar a exposição a consultas externas desnecessárias. Este capítulo aborda como organizar os arquivos principais, isolar serviços recursivos dos

autoritativos e usar a diretiva `forwarders` para controlar de forma segura a resolução DNS de saída.

10.1 — Estrutura modular de configuração segura

Uma abordagem modular ajuda a proteger e gerenciar as configurações com mais facilidade. Em vez de manter tudo em um único arquivo, um layout mínimo e seguro divide a configuração em arquivos separados:

```
/etc/named.conf
├─ options (configurações globais)
├─ logging (controle de log)
├─ acl (listas de controle de acesso)
├─ key (chaves TSIG)
└─ definições de zona
```

`named.conf`

```
include "/etc/named/named.conf.options";
include "/etc/named/named.conf.logging";
include "/etc/named/named.conf.acl";
include "/etc/named/named.conf.keys";
include "/etc/named/named.conf.zones";
```

Cada um desses arquivos pode então ser ajustado individualmente. Um exemplo de `named.conf.options`:

```
options {
    directory "/var/named";
    listen-on port 53 { 192.0.2.53; 127.0.0.1; };
    listen-on-v6 port 53 { none; };
    allow-query { trusted-nets; };
    recursion no;
    dnssec-validation yes;
};
```

Esse bloco `options` configura o comportamento global do servidor BIND 9 com ênfase em segurança e controle:

- `directory "/var/named";` especifica o diretório de trabalho onde arquivos de zona e demais arquivos relacionados são armazenados; todos os caminhos relativos na configuração serão resolvidos a partir dali.
- `listen-on port 53 { 192.0.2.53; 127.0.0.1; };` limita o BIND a escutar apenas nos endereços IPv4 especificados (tipicamente um IP de LAN e o loopback), reduzindo a superfície de ataque ao não expor o serviço DNS em todas as interfaces.

- `listen-on-v6 port 53 { none; };` desabilita completamente a escuta em interfaces IPv6; se o ambiente não usa IPv6, essa é uma boa prática para evitar exposição desnecessária.
- `allow-query { trusted-nets; };` permite consultas DNS apenas de uma ACL predefinida chamada `trusted-nets`, impedindo que clientes não autorizados consultem o servidor.
- `recursion no;` desabilita a recursão, fazendo com que o servidor responda apenas com dados de zonas sob sua autoridade — recomendado para servidores DNS apenas autoritativos, evitando que sejam usados em ataques de amplificação DNS.
- `dnssec-validation yes;` habilita a validação DNSSEC, garantindo que respostas de consultas recursivas sejam verificadas criptograficamente. Mesmo com `recursion` desabilitada aqui, essa configuração reforça a confiança caso a recursão seja habilitada depois, ou em configurações com servidores separados.

named.conf.acl

```
acl "trusted-nets" {  
    127.0.0.1;  
    192.0.2.0/24;  
};
```

Essa ACL também pode ser reutilizada em diretivas `allow-transfer` ou `allow-update`, aplicando políticas de segurança consistentes em toda a configuração.

10.2 — forwarders e separação autoritativo/recursivo

Em muitas redes, especialmente em ambientes corporativos, recomenda-se separar servidores DNS autoritativos de servidores recursivos. Em vez de resolver consultas externas diretamente, o servidor BIND pode encaminhá-las (forward) para resolvers DNS confiáveis — como um provedor upstream, um resolver residente no firewall, ou um serviço de cache local. A diretiva `forwarders` é usada dentro do bloco `options`:

```
options {  
    ...  
    recursion yes;  
    allow-recursion { trusted-nets; };  
    forward only;  
    forwarders {  
        1.1.1.1;  
        8.8.8.8;  
    };  
};
```

Explicação: `recursion yes` habilita a capacidade de resolução recursiva; `allow-recursion` restringe quem pode usá-la; `forward only` instrui o BIND a usar apenas os servidores listados (sem fallback para os servidores raiz); e `forwarders` lista os IPs

dos resolvers upstream. Esse padrão é comum para resolvers internos que atendem clientes internos, mas dependem de resolvers DNS externos para nomes da Internet.

10.3 — Exemplo de separação: um autoritativo e um recursivo

No servidor autoritativo (por exemplo, `ns1.example.com`):

```
options {
    recursion no;
    allow-query { any; };
    allow-transfer { key "mykey"; };
};
```

No servidor recursivo:

```
options {
    recursion yes;
    allow-recursion { 192.168.1.0/24; };
    forward only;
    forwarders {
        192.0.2.53; // o BIND autoritativo ou outro resolver confiável
    };
};
```

Essa separação garante uma divisão clara de papéis, minimiza a superfície de ataque e simplifica o monitoramento e a manutenção.

11. TSIG — Transaction Signatures

Ao configurar um servidor DNS, a segurança é uma preocupação primária. O BIND 9 suporta dois grandes mecanismos para proteger a integridade e a autenticidade dos dados DNS: TSIG (Transaction SIGnatures), para comunicação segura entre servidores DNS ou clientes administrativos, e DNSSEC (DNS Security Extensions), para garantir a validade das respostas DNS vindas de servidores autoritativos. Ambos desempenham papéis essenciais em infraestruturas de DNS modernas e seguras.

11.1 — Gerando e configurando chaves TSIG

O TSIG fornece autenticação de mensagens entre pares DNS. É mais comumente usado para proteger transferências de zona (AXFR) e atualizações dinâmicas realizadas com ferramentas como o `nsupdate`. Para gerar uma chave TSIG:

```
$ sudo tsig-keygen mykey > /etc/named/mykey.key
```

Isso cria um bloco de chave como o seguinte:

```
key "mykey" {  
    algorithm hmac-sha256;  
    secret "f2uZTyrIp+2fYwXLrD6Hx+Z5/LeamYVF3dAP76qSE8E=";  
};
```

11.2 — Zonas seguras com TSIG (master/slave)

Inclua a chave em `named.conf` tanto no servidor master quanto no slave, e autorize seu uso:

Servidor DNS master:

```
include "/etc/named/mykey.key";  
  
zone "example.com" IN {  
    type master;  
    file "db.example.com";  
    allow-transfer { key mykey; };  
    allow-update { key mykey; };  
};
```

Servidor DNS slave:

```
include "/etc/named/mykey.key";  
  
server 192.0.2.1 {  
    keys { mykey; };  
};  
  
zone "example.com" IN {  
    type slave;  
    masters { 192.0.2.1; };  
    file "slaves/db.example.com";  
};
```

11.3 — Atualizações dinâmicas seguras

Para atualizações dinâmicas seguras:

```
$ nsupdate -y hmac-sha256:mykey:7/5QMsuGmt999B0kvZnsY/KM50oW9mmshB91iYeyV9g=  
> server 192.0.2.1  
> update add www.example.com. 3600 IN A 192.0.2.100  
> send
```

O TSIG garante que apenas partes confiáveis possam alterar dados de zona ou solicitar transferências.

12. DNSSEC — Fundamentos

12.1 — O problema que o DNSSEC resolve

O DNSSEC (Domain Name System Security Extensions) reforça a segurança do protocolo DNS tradicional, introduzindo verificação criptográfica dos dados DNS. O problema central que ele resolve é a falta de autenticação nas respostas DNS padrão, o que as torna vulneráveis a ataques como o DNS cache poisoning, em que dados maliciosos são injetados no cache de um resolver, ou ataques de man-in-the-middle, em que as respostas são interceptadas e alteradas. O DNSSEC se defende dessas ameaças garantindo que os dados DNS recebidos possam ser verificados criptograficamente como autênticos.

Importante: o DNSSEC não é um substituto para outros mecanismos de segurança, como SSL/TLS. Ele não criptografa consultas ou respostas DNS, nem oculta o conteúdo dos registros DNS — seu único propósito é a autenticidade: garantir que os dados recebidos realmente vieram do proprietário do domínio e não foram adulterados em trânsito. Diferente de sistemas PKI que exigem cadeias de certificados complexas ou Autoridades Certificadoras (CAs) de terceiros, o DNSSEC opera usando uma cadeia de confiança enraizada na própria hierarquia do DNS, em que cada zona atesta pela próxima.

Uma das grandes vantagens do DNSSEC é ter sido projetado para compatibilidade retroativa: zonas que não são assinadas com DNSSEC continuam funcionando normalmente, e clientes que não suportam DNSSEC continuam resolvendo nomes de domínio sem problemas. Isso torna a adoção incremental e flexível.

12.2 — RRSIG, DNSKEY e o processo de verificação

Isso é realizado por meio de assinaturas digitais adicionadas aos registros DNS em zonas autoritativas. Quando um resolver de cliente consulta um servidor DNS, o servidor inclui não apenas o próprio registro, mas também um RRSIG (Resource Record Signature) correspondente, contendo uma assinatura digital para aquele conjunto de registros. O resolver então usa uma chave pública (publicada por meio de um registro DNSKEY na mesma zona) para verificar se os dados foram assinados por uma autoridade confiável. Se a assinatura for válida, o resolver pode confiar que a informação não foi alterada em trânsito.

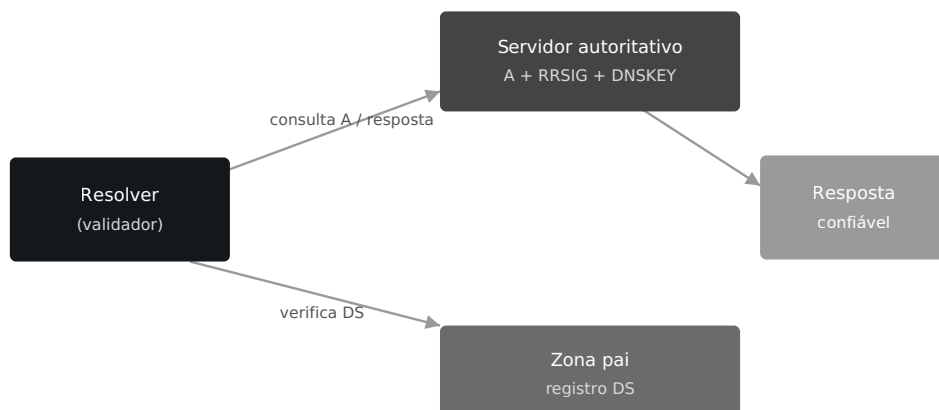


Figura 12.1 — Fluxo simplificado de validação DNSSEC: o resolver obtém o registro e o RRSIG do servidor autoritativo, valida com o DNSKEY e confirma a cadeia de confiança consultando o DS na zona pai.

12.3 — A cadeia de confiança e o registro DS

Do ponto de vista operacional, habilitar a validação DNSSEC nos resolvers recursivos adiciona uma camada crítica de defesa para os usuários, rejeitando respostas forjadas. Enquanto isso, assinar as zonas autoritativas permite que terceiros validem a autenticidade dos seus dados DNS. Essa cooperação mútua fortalece a integridade dos dados DNS em toda a Internet, e conforme a adoção cresce, o DNSSEC desempenha um papel fundamental na construção de uma infraestrutura de rede global mais segura e confiável.

Antes de implantar o DNSSEC em seu servidor autoritativo, é essencial confirmar se sua zona pai suporta DNSSEC. Essa verificação é crítica para estabelecer a cadeia de confiança, que permite que os resolvers DNS validem as assinaturas digitais da sua zona. Esteja ciente de que o administrador da zona pai pode ser diferente do seu registrador de domínio, portanto pode ser necessário contatar uma entidade separada para coordenar o suporte a DNSSEC e a publicação dos seus registros DS.

Para que o DNSSEC funcione corretamente, a zona pai (geralmente gerenciada pelo seu registrador ou registro) precisa publicar um registro especial chamado Delegation Signer (DS). Esse registro atua como um ponteiro criptográfico para a Key Signing Key (KSK) da sua zona.

O registro DS contém um hash da DNSKEY da sua zona (especificamente a KSK), junto com metadados como o algoritmo e o tipo de digest. Ele é publicado na zona pai para estabelecer a cadeia de confiança entre a zona pai e a zona filha. Sem o registro DS na zona pai, os resolvers DNS não conseguem validar seus registros assinados, mesmo que sua zona esteja corretamente assinada. Portanto, depois de gerar sua KSK e assinar sua zona, você deve enviar o registro DS correspondente ao seu registrador, ou ao administrador da zona pai, para publicação.

13. Habilitando DNSSEC com KASP

O BIND oferece diferentes abordagens para gerenciar assinaturas DNSSEC ao longo do ciclo de vida de uma zona. O método mais recomendado é o sistema totalmente automatizado baseado em Key and Signing Policy (KASP). Essa abordagem automatiza a geração de chaves criptográficas, o processo de rollover e a assinatura da zona, minimizando erro humano e garantindo segurança consistente.

13.1 — dnssec-policy e inline-signing

Key and Signing Policy (KASP) é um framework de configuração que define como as chaves de assinatura DNSSEC são gerenciadas e como a zona é assinada ao longo do tempo. É a abordagem recomendada para implantação e manutenção totalmente automatizadas do DNSSEC. Na maioria dos casos, os administradores podem confiar na política padrão embutida, que segue as melhores práticas atuais para assinatura de zona segura e confiável.

Para habilitar a assinatura DNSSEC automática usando KASP, defina a zona com a seguinte configuração:

```
zone "example.com" {  
    type primary;  
    file "db.example.com";  
    dnssec-policy default;  
    inline-signing yes;  
};
```

O statement `dnssec-policy` é suficiente para gerar automaticamente as chaves de assinatura necessárias, junto com os registros DNSKEY, RRSIG e NSEC da zona. O BIND também cuidará das tarefas contínuas de manutenção do DNSSEC, como renovar assinaturas que estão expirando e realizar rollovers de chave de forma transparente.

A política DNSSEC `default` gera um único par de chaves que assina a zona inteira; ela também usa registros NSEC para fornecer negação de existência autenticada, permitindo que resolvers verifiquem com segurança quais registros estão ausentes. Essa política segue as melhores práticas atuais e é adequada para a maioria das implantações, raramente exigindo personalização.

O recurso `inline-signing` no BIND permite habilitar DNSSEC para uma zona sem modificar manualmente o arquivo de zona original. Em vez de substituir o arquivo de zona não assinado, o BIND cria e mantém internamente uma versão assinada dele. A diretiva `inline-signing yes` instrui o BIND a ler o arquivo de zona original, não assinado (como `db.example.com`), gerar os registros DNSSEC necessários — incluindo DNSKEY, RRSIG e NSEC ou NSEC3 — e gravar uma versão assinada do arquivo de zona (por exemplo, `db.example.com.signed`) automaticamente. O BIND monitora

continuamente o arquivo não assinado em busca de mudanças e regenera as assinaturas de forma transparente sempre que os registros são atualizados ou quando assinaturas existentes se aproximam da expiração. Esse mecanismo permite que os administradores mantenham suas zonas em texto plano, enquanto o BIND cuida da assinatura DNSSEC em segundo plano.

Depois de habilitar `dnssec-policy` e `inline-signing`, use `rndc reload` para aplicar a nova configuração de zona:

```
$ sudo rndc reload
```

13.2 — Verificando o status de assinatura

Para verificar o status de assinatura de uma zona como `example.com`, use:

```
$ sudo rndc dnssec -status example.com
dnssec-policy: default
current time: Thu Jul 3 02:55:11 2025

key: 46460 (ECDSAP256SHA256), CSK
  published:      yes - since Thu Jul 3 01:40:18 2025
  key signing:    yes - since Thu Jul 3 01:40:18 2025
  zone signing:   yes - since Thu Jul 3 01:40:18 2025

No rollover scheduled
- goal:          omnipresent
- dnskey:        rumoured
- ds:            hidden
- zone rrsig:    rumoured
- key rrsig:     rumoured
```

Nesse ponto, sua zona está corretamente assinada usando DNSSEC, e o BIND está mantendo as assinaturas automaticamente graças à `dnssec-policy` e ao `inline-signing` configurados. No entanto, para que resolvers externos consigam validar os dados da sua zona, a cadeia de confiança precisa ser completada — isto é, o registro DS que acabou de ser gerado precisa ser publicado na zona pai (geralmente no registrador). Somente após essa etapa os resolvers DNS conseguirão verificar a autenticidade das assinaturas da sua zona.

13.3 — Gerando e publicando o registro DS

Para gerar um registro DS a partir de uma Key Signing Key (KSK) DNSSEC, é possível usar o comando `dnssec-dsfromkey`. Esse registro é essencial para estabelecer a cadeia de confiança entre sua zona DNS e sua zona pai.

Para garantir que você está referenciando o arquivo DNSKEY correto ao gerar o registro DS, primeiro liste todos os arquivos de chave pública no diretório da zona:

```
$ sudo ls /var/named/Kexample.com.*.key
/var/named/Kexample.com.+013+46460.key
```

Em seguida, execute o comando a seguir para gerar o registro DS com base no arquivo de KSK pública selecionado:

```
$ dnssec-dsfromkey /var/named/Kexample.com.+013+46460.key
example.com. IN DS 46460 13 2
A4C3950B2297A769C1BBE929DE93DB93A94E0177B4992266F9DC35F4A923AC7F
```

Os campos representam: `46460` — o Key Tag (usado pelos resolvers para identificar qual chave usar); `13` — o número do algoritmo (ECDSAP256SHA256, nesse caso); `2` — o tipo de digest (2 = SHA-256); e o campo final é o digest real da chave pública. Esse registro DS deve ser enviado ao seu registrador de domínio ou operador da zona pai, para que eles possam publicá-lo, completando a cadeia de confiança.

Após publicar o registro DS no registrador (ou seja, na zona pai), você pode verificar sua presença e propagação usando o seguinte comando:

```
$ sudo rndc dnssec -checkds published example.com
```

Esse comando consulta a zona pai para verificar se o registro DS correspondente à sua KSK está visível. Se o registro estiver corretamente publicado, o BIND atualizará o status da chave para omnipresent, indicando que a cadeia de confiança está completa.

Essa etapa é crucial ao usar assinatura automática com uma política DNSSEC (`dnssec-policy`) no BIND. O rollover de chave só prosseguirá depois que o BIND confirmar que a chave antiga foi completamente confiada e propagada. Se o registro DS ainda não for detectado, a chave permanecerá no estado `rumoured` e não será removida ou substituída até que a validação seja bem-sucedida.

13.4 — Consultando dados DNSSEC com dig

Para realizar uma consulta DNS ao servidor local e solicitar dados relacionados a DNSSEC em formato legível:

```
$ dig @127.0.0.1 example.com A +dnssec +multiline
```

Explicando cada parte: a flag `+dnssec` solicita informações DNSSEC na resposta — especificamente, inclui registros RRSIG, que contêm as assinaturas digitais do conjunto de registros consultado. A flag `+multiline` formata a saída em múltiplas linhas, facilitando a leitura e a inspeção dos campos da assinatura criptográfica.

Essa consulta é útil para verificar se o DNSSEC está corretamente configurado no seu servidor autoritativo. Uma resposta válida incluirá o registro A assinado, junto com uma entrada RRSIG, comprovando que o registro foi assinado criptograficamente. Se nenhuma assinatura estiver presente, o DNSSEC não está corretamente habilitado, ou a chave ainda não é confiável para o resolver.

14. DANE — Autenticação de certificados via DNS

14.1 — Conceito e casos de uso

DANE (DNS-based Authentication of Named Entities) se apoia no DNSSEC para associar certificados X.509 a nomes de domínio. Ele permite que administradores publiquem, de forma segura, informações de certificado no DNS usando registros TLSA.

Isso viabiliza diversos cenários importantes para reforçar a segurança online. Uma aplicação-chave é a capacidade de validar certificados SSL/TLS sem depender exclusivamente de Autoridades Certificadoras (CAs) tradicionais. Como o DANE usa o DNSSEC para verificar a autenticidade das informações de certificado, ele mitiga o risco de CAs comprometidas ou fraudulentas emitirem certificados falsos.

Outro caso de uso crítico é reforçar políticas estritas de segurança de transporte para serviços como SMTP, HTTPS ou XMPP. Usando DANE, administradores podem publicar expectativas sobre o certificado que um serviço deve apresentar, garantindo que os clientes só aceitem conexões de servidores devidamente autenticados. Além disso, o DANE permite a publicação segura de chaves públicas conhecidas ou hashes de certificado, ajudando a detectar e prevenir certificados fraudulentos ou não autorizados, ao permitir que os clientes comparem o certificado apresentado pelo servidor com os valores confiáveis obtidos via DNSSEC — criando, assim, uma âncora de confiança forte, enraizada no DNS.

Por exemplo, um administrador de servidor de e-mail pode publicar um registro TLSA:

```
_25._tcp.mail.example.com. IN TLSA 3 1 1 <hash-do-certificado>
```

Isso indica que o servidor de e-mail na porta 25 deve usar um certificado que corresponda ao hash publicado; e, como o registro TLSA é protegido pelo DNSSEC, sua autenticidade é garantida.

14.2 — Publicando um registro TLSA

Por exemplo, para publicar um registro TLSA para `https://www.example.com` usando DANE, siga estes passos. Supondo que o servidor já esteja configurado com TLS, use o `openssl` para extrair o certificado:

```
$ openssl s_client -connect example.com:443 -showcerts </dev/null \  
| openssl x509 -outform DER > cert.der
```

Agora use o `openssl` para gerar o registro TLSA com base no certificado extraído anteriormente em formato DER:

```
$ openssl x509 -in cert.der -inform DER -noout -pubkey \  
| openssl pkey -pubin -outform DER \  
| openssl dgst -sha256
```

Saída:

```
SHA2-256(stdin)= d819bae6c43(...)
```

Esse comando executa os seguintes passos: extrai a chave pública do arquivo `cert.der`; converte-a para formato DER usando o utilitário `openssl pkey`; e calcula seu hash SHA-256, exigido para um registro TLSA com matching type 1 (hash do SubjectPublicKeyInfo).

O hash resultante `d819bae6c4(...)` é usado como a parte final do registro TLSA:

```
_443._tcp.www.example.com. IN TLSA 3 1 1 <HASH_SHA256>
```

Substitua `<HASH_SHA256>` pelo hash real impresso pelo `openssl`. Ao adicionar esse registro à sua zona DNS, você está publicando um registro DANE que vincula o serviço HTTPS (porta 443) de `example.com` a uma chave pública específica (identificada por seu hash SHA-256). Isso permite que resolvers DNS e clientes que suportam DANE validem o certificado TLS apresentado pelo servidor, sem depender exclusivamente de Autoridades Certificadoras (CAs) tradicionais.

Publicar um registro TLSA com DANE, protegido pelo DNSSEC, ajuda a prevenir ataques de man-in-the-middle, emissão fraudulenta por CAs e falsificação de certificado. Isso garante que apenas o certificado ou a chave pública explicitamente definidos no DNS sejam confiáveis, mesmo que uma CA seja comprometida — reforçando o TLS ao vincular a validação de certificado ao DNS, e reduzindo a dependência de autoridades certificadoras de terceiros.

15. Cheat-sheet de comandos

Comando	Função
<code>named-checkconf</code>	Valida a sintaxe do arquivo <code>named.conf</code> e dos arquivos incluídos.
<code>named-checkconf -z -p</code>	Valida também as zonas referenciadas e imprime a configuração em forma canônica.
<code>named-checkzone <zona> <arquivo></code>	Valida a sintaxe e a consistência de um arquivo de zona.
<code>named-compilezone -f text -F raw -o <saída> <zona> <arquivo></code>	Compila um arquivo de zona de texto para o formato binário raw.
<code>rndc reload</code>	Recarrega toda a configuração e todas as zonas.
<code>rndc reload <zona></code>	Recarrega uma zona específica.
<code>rndc status</code>	Exibe o status operacional do servidor.
<code>rndc flush</code>	Limpa todos os caches do resolver.
<code>rndc stop</code>	Encerra o BIND de forma graciosa.
<code>rndc sync / freeze / thaw</code>	Sincroniza, congela ou libera a gravação de zonas dinâmicas (journal .jnl) no arquivo de zona.
<code>rndc-confgen -a</code>	Gera uma chave rndc local em <code>/etc/rndc.key</code> .
<code>rndc dnssec -status <zona></code>	Mostra o status de assinatura DNSSEC/KASP de uma zona.
<code>rndc dnssec -checkds published <zona></code>	Verifica se o registro DS foi publicado na zona pai.
<code>tsig-keygen <nome></code>	Gera uma chave TSIG (HMAC) para transferências e atualizações seguras.
<code>dnssec-dsfromkey <arquivo.key></code>	Gera o registro DS a partir de uma KSK.
<code>nsupdate -y <algoritmo:chave:segredo></code>	Realiza atualizações dinâmicas de DNS autenticadas por TSIG.
<code>dig @<servidor> <domínio> <tipo></code>	Consulta um servidor DNS específico por um tipo de registro.
<code>dig +noall +answer</code>	Exibe apenas a seção de resposta da consulta.
<code>dig +dnssec +multiline</code>	Solicita registros RRSIG e formata a saída em várias linhas.

<code>host <domínio> [servidor]</code>	Consulta simples e direta a um domínio, opcionalmente via um servidor específico.
<code>openssl s_client -connect <host>:<porta> -showcerts</code>	Extraí o certificado TLS apresentado por um serviço, para gerar um registro TLSA (DANE).

16. Exercícios comentados

16.1 — O papel de uma zona na arquitetura DNS

ENUNCIADO

Qual é o papel de uma zona na arquitetura DNS?

RESPOSTA

Uma zona representa uma porção do espaço de nomes DNS gerenciada por um servidor DNS autoritativo específico. Ela contém registros DNS (A, NS, MX, etc.) para domínios e subdomínios, e é armazenada em um arquivo de zona. Uma zona primária é gravável e mantida no servidor master; uma zona secundária é uma réplica somente leitura, mantida em um servidor slave.

16.2 — Configurando uma zona secundária

ENUNCIADO

Como configurar uma zona do BIND como secundária?

RESPOSTA

Declarando o tipo da zona como `slave`, especificando o caminho do arquivo para armazenar os dados replicados e definindo o endereço IP do servidor master:

```
zone "example.com" IN {  
    type slave;  
    file "slaves/example.com.zone";  
    masters { 192.0.2.1; };  
};
```

16.3 — Verificando a sintaxe do named.conf

ENUNCIADO

Qual comando é usado para verificar a sintaxe do arquivo named.conf?

RESPOSTA

Use o comando `named-checkconf` :

```
$ sudo named-checkconf
```

Ele não retorna nenhuma saída se a configuração for válida; erros são impressos com o número da linha.

16.4 — Recarregando zonas sem reiniciar o serviço

ENUNCIADO

Como usar o comando `rndc` para recarregar os arquivos de zona DNS sem reiniciar o serviço BIND?

RESPOSTA

Basta usar o comando `rndc reload` :

```
$ sudo rndc reload
```

16.5 — Consultando o registro SOA com dig

ENUNCIADO

Como consultar o registro `SOA` de `example.com` usando o `dig` ?

RESPOSTA

Especifique o tipo de registro explicitamente:

```
$ dig example.com SOA
```

Saída de exemplo:

```
;; QUESTION SECTION:
;example.com.                IN      SOA

;; ANSWER SECTION:
example.com.                 3600    IN      SOA      ns.icann.org. noc.dns.icann.org.
2025011737 7200 3600 1209600 3600
```

16.6 — Criando um arquivo de zona completo

ENUNCIADO

Crie um arquivo de zona chamado `db.example.com`, contendo registros `SOA`, `NS`, `A`, `AAAA`, `MX`, `CNAME` e `TXT` para o domínio `example.com`. Use `@` para a origem e defina um TTL de 3600 segundos.

RESPOSTA

```
$TTL 3600
@ IN SOA ns1.example.com. admin.example.com. (
    2025080201 ; serial
    3600 ; refresh
    1800 ; retry
    1209600 ; expire
    86400 ) ; minimum
    IN NS ns1.example.com.
    IN NS ns2.example.com.
ns1 IN A 192.0.2.1
ns2 IN A 192.0.2.2
www IN A 192.0.2.10
www6 IN AAAA 2001:db8::10
mail IN A 192.0.2.20
@ IN MX 10 mail.example.com.
ftp IN CNAME www.example.com.
@ IN TXT "v=spf1 +mx -all"
```

Em seguida, valide com `named-checkzone` :

```
$ sudo named-checkzone example.com db.example.com
zone example.com/IN: loaded serial 2025080201
OK
```

16.7 — Criando uma zona reversa

ENUNCIADO

Crie um arquivo de zona reversa para a rede `192.0.1.0/24`, definindo registros `PTR` para os hosts da sua zona direta.

RESPOSTA

```
$TTL 3600
@ IN SOA ns1.example.com. admin.example.com. (
    2025080201 ; serial
    3600 ; refresh
    1800 ; retry
    1209600 ; expire
    86400 ) ; minimum
    IN NS ns1.example.com.
    IN NS ns2.example.com.
1 IN PTR ns1.example.com.
2 IN PTR ns2.example.com.
10 IN PTR www.example.com.
20 IN PTR mail.example.com.
```

16.8 — Autorizando atualizações dinâmicas

ENUNCIADO

Gere uma chave TSIG com o `tsig-keygen`, e configure `named.conf` para permitir atualizações dinâmicas usando essa chave.

RESPOSTA

```
$ tsig-keygen mykey
key "mykey" {
    algorithm hmac-sha256;
    secret "Lu1WX86E30iPhX6pfqQNHvQNNeJYsQLHBpnLAG8g26g=";
};
```

```
key "mykey" {
    algorithm hmac-sha256;
    secret "Lu1WX86E30iPhX6pfqQNHvQNNeJYsQLHBpnLAG8g26g=";
};

zone "example.com" IN {
    type master;
    file "db.example.com";
    allow-update { key "mykey"; };
};
```

E adicione um host dinamicamente:

```
$ nsupdate -y hmac-sha256:mykey:Lu1WX86E30iPhX6pfqQNHvQNNeJYsQLHBpnLAG8g26g=
> server 127.0.0.1
> update add ftp.example.com. 3600 IN A 192.0.2.55
> send
```

16.9 — Encaminhando consultas para resolvers públicos

ENUNCIADO

Escreva um trecho de configuração para encaminhar todas as consultas DNS para `8.8.8.8` e `1.1.1.1`.

RESPOSTA

```
options {
    forwarders {
        8.8.8.8;
        1.1.1.1;
    };
};
```

16.10 — Habilitando a política DNSSEC padrão

ENUNCIADO

Como configurar o BIND para usar a política DNSSEC padrão automaticamente?

RESPOSTA

```
zone "secure.example.com" {  
    type primary;  
    file "secure.example.com.db";  
    dnssec-policy default;  
};
```

16.11 — Gerando o registro DS e consultando dados DNSSEC

ENUNCIADO

Qual comando gera um registro DS a partir de um arquivo de chave, e qual consulta DNS retorna dados DNSSEC?

RESPOSTA

```
$ sudo dnssec-dsfromkey /var/named/Kexample.com.+013+46460.key  
$ dig A example.com +dnssec +multiline
```

16.12 — Slave com TSIG

ENUNCIADO

Configure um servidor BIND slave que use autenticação TSIG para transferências de zona a partir do master. Escreva a configuração necessária em ambos os servidores.

RESPOSTA

No servidor master (named.conf):

```
key "transfer-key" {
    algorithm hmac-sha256;
    secret "nt1prljbkkyeLEB9SfQ0vljapbAuy6NXvKtbYn8Dhao=";
};

server 192.0.2.10 {
    keys { transfer-key; };
};

zone "example.com" {
    type master;
    file "db.example.com";
    allow-transfer { key transfer-key; };
};
```

No servidor slave (named.conf):

```
key "transfer-key" {
    algorithm hmac-sha256;
    secret "nt1prljbkkyeLEB9SfQ0vljapbAuy6NXvKtbYn8Dhao=";
};

server 192.0.2.1 {
    keys { transfer-key; };
};

zone "example.com" {
    type slave;
    masters { 192.0.2.1 key transfer-key; };
    file "slaves/db.example.com";
};
```

Essa configuração estabelece uma transferência de zona segura entre um master e um slave usando TSIG. Ambos os servidores definem uma chave compartilhada chamada `transfer-key`, usando o algoritmo `hmac-sha256`. O master é configurado para permitir transferências apenas a servidores autenticados com essa chave, enquanto o slave é configurado para usar a mesma chave ao contatar o master, garantindo que a transferência seja autenticada e protegida contra adulteração ou spoofing.

16.13 — Verificando um registro TLSA (DANE)

ENUNCIADO

Qual consulta você usaria para verificar o registro TLSA do DANE de um serviço HTTPS na porta 443?

RESPOSTA

```
$ dig _443._tcp.example.com TLSA +dnssec
```

O prefixo `_443._tcp.` segue a convenção de nomenclatura do DANE, em que `_porta._protocolo.domínio` identifica o serviço. A flag `+dnssec` instrui o `dig` a solicitar dados relacionados a DNSSEC, incluindo a assinatura RRSIG, permitindo ao cliente verificar se o registro TLSA foi validado criptograficamente e faz parte de uma cadeia confiável.

16.14 — Anycast nos servidores raiz

ENUNCIADO

O que é roteamento anycast, e por que ele é usado nos servidores raiz?

RESPOSTA

O roteamento anycast permite que vários servidores em diferentes localizações geográficas compartilhem o mesmo endereço IP; as consultas DNS são automaticamente roteadas para a instância mais próxima. Os servidores raiz usam anycast para melhorar a disponibilidade global, reduzir a latência e oferecer tolerância a falhas, garantindo que as consultas cheguem ao servidor saudável mais próximo.

16.15 — Reputação de servidores de e-mail e DNS reverso

ENUNCIADO

Como a resolução DNS reversa afeta a reputação de servidores de e-mail?

RESPOSTA

O DNS reverso é crítico para servidores de e-mail, porque muitos sistemas anti-spam verificam se o endereço IP remetente possui um registro PTR válido, correspondente ao domínio informado nos cabeçalhos da mensagem. Se não existir um registro PTR, ou se ele não corresponder, a mensagem pode ser marcada como spam ou rejeitada.

16.16 — Riscos de habilitar recursion yes em um servidor público

ENUNCIADO

Quais são as implicações de habilitar `recursion yes;` em um servidor DNS voltado à Internet pública?

RESPOSTA

Habilitar recursão em um servidor DNS público pode expô-lo a abusos, como ataques de amplificação DNS, em que o servidor é usado para inundar uma vítima com tráfego. Servidores recursivos públicos devem implementar limitação de taxa, controle de acesso via `allow-recursion`, ou evitar a recursão completamente (`recursion no;`), a menos que sejam especificamente destinados a esse papel.

17. Resumo geral

- O **DNS** é um sistema hierárquico e distribuído que traduz nomes de domínio em endereços IP, organizado em raiz, TLDs, domínios de segundo nível e subdomínios, com autoridade delegada por meio de zonas.
- Uma zona pode ser **primária** (master, gravável) ou **secundária** (slave, réplica somente leitura sincronizada por AXFR/IXFR); servidores **recursivos** resolvem consultas em nome dos clientes, e **forwarders** encaminham consultas para resolvers upstream.
- **Zonas reversas**, sob `in-addr.arpa` (IPv4) e `ip6.arpa` (IPv6), mapeiam endereços IP de volta a nomes de domínio usando registros `PTR`.
- O **BIND 9** é configurado centralmente pelo arquivo `named.conf`, estruturado em statements (`options`, `zone`, `acl`, `include`, `logging`), cada um delimitado por chaves e terminado com ponto e vírgula.
- Arquivos de zona seguem o formato `[nome] [TTL] [classe] [tipo] [dado]`, começam com um registro `SOA` e incluem registros como `NS`, `A`, `AAAA`, `MX`, `CNAME`, `TXT` e `PTR`; nomes sem ponto final são relativos à origem da zona.
- O **número de série** do SOA deve ser incrementado a cada alteração da zona, para que os servidores secundários detectem a atualização e sincronizem os dados.
- `named-checkconf` valida a sintaxe de `named.conf`; `named-checkzone` valida a sintaxe e a consistência dos arquivos de zona antes de colocá-los em produção.
- O `rndc` permite administrar o BIND em execução — recarregar zonas, verificar status, limpar cache e sincronizar journal files — sem reiniciar o serviço.
- `dig` e `host` são as ferramentas padrão para consultar e depurar respostas DNS, com o `dig` oferecendo saída mais detalhada.
- Hosts podem ser adicionados manualmente nos arquivos de zona, ou dinamicamente com `nsupdate`, autenticado por **TSIG**; atualizações dinâmicas são gravadas em um journal file (`.jnl`) antes de serem mescladas ao arquivo de zona.
- O formato `masterfile-format raw`, gerado com `named-compilezone`, acelera o carregamento de zonas grandes; a **delegação de zonas** distribui a administração de subdomínios para outros servidores DNS por meio de registros `NS`.
- **dnsmasq** é indicado para redes pequenas, embarcadas e containerizadas; **PowerDNS** é indicado para ambientes de grande escala com backends de banco de dados e APIs de automação.

- A segurança do BIND começa por limitar interfaces de escuta (`listen-on`), restringir consultas por ACL (`allow-query`), separar papéis autoritativo/recursivo e usar `forwarders` para consultas externas.
- **TSIG** autentica transferências de zona e atualizações dinâmicas usando chaves compartilhadas (HMAC); **DNSSEC** garante a autenticidade e a integridade dos dados DNS por meio de assinaturas digitais (`RRSIG` / `DNSKEY`) e de uma cadeia de confiança ancorada no registro `DS` da zona pai.
- O **KASP** (`dnssec-policy` + `inline-signing`) automatiza a geração de chaves, a assinatura de zonas e o rollover, sendo a abordagem recomendada para DNSSEC no BIND 9 moderno.
- **DANE** estende o DNSSEC para publicar e validar certificados TLS via registros `TLSA`, reduzindo a dependência exclusiva de Autoridades Certificadoras tradicionais.